SOTER: Guarding Black-box Inference for General Neural Networks at the Edge

Anonymous Author(s)

Abstract

Tremendous AI applications powered by deep learning inference are increasingly deployed on untrusted edge devices to provide an interactive user experience. This necessitates securely offloading the proprietary model from the cloud to the edge, and using a co-located accelerator (e.g., GPU) to speed up model inference locally. Due to the lack of commercial secure GPUs, a prevalent approach is to use Trusted Execution Environments (TEEs) to protect model confidentiality from the untrusted software stack. However, despite much effort, securely outsourcing inference computations from a TEE to a faster, yet untrusted GPU remains a fundamental challenge. Existing TEE-based solutions either ban the choice of GPU acceleration or take a handcrafted approach to partition insensitive model layers to run in a GPU. While the latter approach achieves better performance, it requires substantial expertise for model partitioning and is only safe for specific models.

We propose SOTER, the first system that protects confidentiality while achieving low inference latency for general DNN models. Our key observation is that there is often an *associativity* property among many inference operators in a model. Therefore, SOTER automatically profiles and transforms a major fraction of operators into *parameter-morphed* and *confidentiality-preserving* operators to execute in an untrusted GPU, and fully recovers execution results into accurate results within TEE. Based on these steps, SOTER further designs an *oblivious fingerprinting* technique to safely detect integrity breaches of these morphed operators outside TEE. Extensive evaluation of SOTER and two notable systems with nine prevalent models in four most popular categories shows that: SOTER achieves a high confidentiality score in terms of a well-known metric and can effectively detect integrity breaches with high probability; SOTER achieves comparable performance with handcrafted partition-based systems.

1 Introduction

Driven by the remarkable success of Deep Learning (DL) [25], mission-critical applications are trending to utilize Deep Neural Networks (DNNs) to help analyze real-time data by taking inputs (e.g., images) and transforming those inputs into actionable insights (i.e., inference). For instance, autopilot navigation [8, 13], home monitoring [17] and visual assistance [60] are pervasive examples powered by DNN model (in short, model) inference.

More and more inference services are run natively on edge devices for two reasons. First, numerous mission-critical applications require *millisecond level* responses to conduct real-time data analysis [31, 85], which makes a cloud-side approach unsuitable because clients have to transfer a high volume of queries over the network [55]. Second, since clients' queries can be sensitive (e.g., images for face recognition), existing cloud-side approaches leverage homomorphic encryption [23, 30] and Garbled Circuits [4] to preserve query confidentiality, coming at substantial computational overhead. For example, in Delphi [55], a state-of-the-art cloud-side secure inference system, making one query requires ∼60MB communication and ∼4s latency, violating the stringent latency requirement of real-time applications.

However, offloading inference services from the cloud to the edge poses a threat to model confidentiality. As obtaining an accurate model requires substantial resources and efforts to design an optimal model architecture and train model parameters with enormous training data [57], if directly offloading a model to the edge with *white-box* accesses, a corrupted client (e.g., business competitor) can directly observe and steal the proprietary model architecture and parameters [54, 91].

Hence, there is an urgent demand to achieve low inference latency while preserving model confidentiality with *black-box* accesses at the edge. *Black-box* means that a client can query the model as an oracle but will learn nothing about the proprietary model architecture and parameters.

Although there are some promising trusted GPU research systems [5, 87] that can protect model confidentiality and accelerate inference with the strong GPU computation power, these systems are not commonly available as they require extensive hardware modifications [87]. Thus, in this paper, we consider only an untrusted commercial CPU and GPU.

A recent pragmatic approach is to leverage Intel SGX [12], a pervasively used Trusted Execution Environment (TEE) to host the offloaded model. SGX provides code integrity and data confidentiality for enclaves, such that the offloaded model hosted in an enclave can not be seen or tampered with from other untrusted software stack running on the client's platform [7, 56]. Moreover, by leveraging the attestation tool of SGX, the model provider can perform remote attestation to the client and ensure that the correct model is safely running at the client device equipped with SGX [12].

Nevertheless, SGX is not a panacea to solve the problem. Existing SGX-based approaches are facing a fundamental

confidentiality dilemma between black-box access and fast execution: either an approach has to enclose all inference computations within enclave to preserve model confidentiality, or outsource part of the sensitive model to the untrusted GPU (with white-box access) for fast execution. As a typical work, MLcapsule [33] hosts the entire model layers in enclaves to guarantee model confidentiality. As a fully TEE-shielded system without involving GPU, MLcapsule achieves unsatisfactory performance (up to 22.6% latency explosion, shown in §6) for diverse inference workloads.

Confronted with the confidentiality dilemma, known remedies including eNNclave [70] and DeepEnclave [32] take a partition-based approach to leverage the strong computation power of a co-located GPU. Specifically, these approaches require manually partitioning the offloaded model into public and private layers. By holding private layers in enclave and partitioning public layers to GPU, sensitive parameters in private layers remain confidential while the untrusted GPU can handle the public layers fastly, achieving lower inference latency than fully TEE-shielded systems (e.g., MLcapsule).

Although existing partition-based solutions can greatly accelerate inference by engaging the GPU power, but in general, these handcrafted approaches are error-prone and not secure enough to support general DNN models. In more detail, for specific models with simple and well-learned architectures (e.g., ResNet [82] and VGG [67]), existing solutions can be error-free [70]. This is because, these simple models are proven to have a lucid sensitivity boundary among layers, i.e., using an explicit set of extraction layers (with sensitive parameters) to distill input features, and use other layers (with less sensitive parameters) to perform class-specific transformations on intermediate results [37,81,92]. Hence, it is secure for a handcrafted approach to keep all explicit sensitive layers within enclave and partition other layers to GPU.

Unfortunately, existing handcrafted model partition approaches are error-prone to big models. This is because, with the rapid advancement of AI optimization techniques (e.g., Neural Architecture Search (NAS) [9, 19]), models are evolved to have complex architectures composed of numerous layers with intricate inter-layer linkages [47, 62, 97]. Due to the weak explainability of big models' complex architectures [20], a handcrafted model partition approach is error-prone to identify the real sensitive layers thus potentially causes leakage of confidentiality (shown in §5.1.2).

Our key observation to securely partition general models is the *associativity* property of many inference operators. *Associativity* means that the way that factors are grouped in an operator's computation does not change the final result [53]. Hence, we can securely transform associative operators into *parameter-morphed* operators to execute in GPU without revealing the plaintext data, and then recover the execution results in enclave without losing precision. Moreover, since most computational-intensive inference operators are associative (e.g., *Convolution*, *Linear*), we can securely partition a major fraction of inference computations to GPU, which significantly reduces inference latency.

Based on this observation, we propose SOTER, the first partition-based approach that provides stronger confidentiality guarantees to general models at the edge. SOTER carries a *Morph Then Recover* (MTR) protocol in enclave to securely partition some inference computations to GPU. Specifically, SOTER profiles the entire model and morphs a fraction of associative operators with randomly generated *blinding coins* (scalars) in enclave, partitions these *parameter-morphed* operators to securely execute in GPU, and fully *recovers* the accurate inference results with reciprocal coins maintained in enclave. With the MTR protocol, all partitioned operators' parameters are hidden without revealing the plaintext while the inference can be accelerated by engaging the GPU power.

Even with the MTR protocol, SOTER still faces an integrity challenge in the presence of a third-party attacker at the edge. Specifically, the client device might be hijacked by a third-party attacker who can observe and manipulate any components outside enclave (e.g., morphed parameters in GPU), misleading a model to produce wrong outputs to drop the inference accuracy and causing application failures.

To tackle the integrity challenge, SOTER develops an *oblivious fingerprint* technique to detect integrity breaches outside enclave in a *challeng-proof* manner. Specifically, before the inference, SOTER collects several fingerprints of partitioned operators which hold the ground-truth correctness of these operators' execution results; Then, during the inference, SOTER randomly challenges partitioned operators to ask them for execution proofs. By comparing the returned proofs with expected fingerprint outputs in enclave, SOTER can learn whether an integrity breach happens.

To prevent an adversary distinguishing fixed fingerprint challenges and bypassing integrity detection, stemming from the same associativity observation, SOTER dynamically derives new fingerprints at runtime that statistically share the same distribution as the normal input to partitioned operators (§5.2), which makes fingerprint challenges oblivious.

We implemented SOTER on Slalom [83] codebase, a mature and lightweight framework for writing neural networks. We evaluated SOTER with nine state-of-the-art DNN models covering four prevalent types: Multi-layer Perception (MLP), Convolution Neural Network (CNN), Recurrent Neural Network (RNN) and Transformer. We compared SOTER to two notable SGX-based secure inference systems (MLcapsule [33] and eNNclave [70]). Our evaluation shows that:

- **Security:** SOTER provided stronger confidentiality guarantees to general DNN models while existing handcrafted partition system (eNNclave) only protected specific models in terms of a well-known security metric; SOTER can quickly detect integrity breaches of partitioned operators outside enclave with overwhelming high probability (§5.2).
- **Performance:** SOTER was efficient. SOTER's inference

latency was 3.5×∼22.6× lower than the pure TEE-shielded baseline (MLcapsule) and has up to 1.8× latency reduction over the handcrafted partition baseline (eNNclave) (§6.1).

• **Accuracy:** SOTER was accurate. SOTER can achieve the ground-truth accuracy of the original model (same as MLcapsule), while existing partition-based baseline (eNNclave) caused 1.1%∼6.3% accuracy drops (§6.2).

Our major contribution is the MTR protocol, the first work that automatically provides stronger confidentiality and integrity guarantees to general DNN models' inference at the edge. Compared to existing relevant systems, SOTER's security gains are two-fold: for confidentiality, SOTER can achieve the same guarantee as the handcrafted partition-based systems for all their evaluated specific models, and SOTER provides stronger guarantee for general and complex big models (§5.1), which are infeasible to handcraft in these exiting systems; for integrity, SOTER can effectively detect integrity attacks outside enclaves with high probability, while all existing partition-based systems are not designed to handle such integrity attacks (§5.2).

Furthermore, our extensive evaluation on prevalent models shows that SOTER achieves reasonable performance overhead, which is comparable with handcrafted systems and much better than existing TEE-shielding systems; SOTER also achieves high accuracy, which is the same as insecure inference and much higher than handcrafted systems. This makes SOTER unique to greatly promote general security-critical AI applications and to effectively protect diverse well-trained proprietary DNN models at the edge, including home monitoring [17], visual assistance [60] and autopilot navigation [8]. SOTER's source code is released on `github.com/ndss22p16/soter`.

In the remaining of this paper, §2 introduces background. §3 gives an overview of SOTER. §4 describes SOTER's design and security analysis. §6 shows our evaluation. §7 discusses related work and §8 concludes.

2 Background and Related Work

2.1 Deep Neural Network

As shown in Figure 1, a DNN model (in short, model) can be represented as a sequence of connected *layers* with each layer assigned a set of *operators*. An operator is either a *linear operator* or a *nonlinear operator* where a linear operator is weighted by a set of pre-trained *parameter matrices* (in short, parameters).

The life-cycle of a model can be divided into two phases: a training phase and an inference phase. In the training phase, to obtain an accurate model, the model provider has to design an optimal model architecture (e.g., through neural architecture search [9,19]), passes the model with a private training dataset multiple times, and updates the numerical values of parameters by calculating the gradients with forward-backward propagations [57]. Both the model architecture and parameters are sensitive properties as they require substantial expertise knowledge and training efforts [79]. In the inference phase, a model takes input data and passes the data through all operators with fixed parameters. Without losing generality, we use image classification as an example to illustrate the model composition in next discussions.

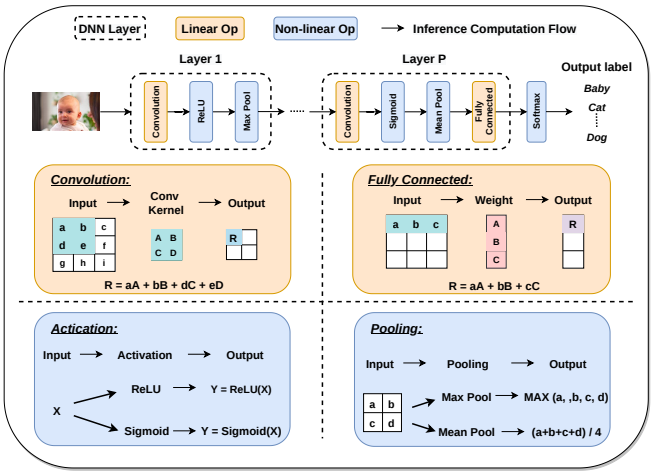


Figure 1: An overview of a DNN model.

Inference workflow. Figure 1 shows the workflow of running an inference on a pre-trained model with pre-designed architecture and pre-trained parameters. A model M takes an input X (e.g., an image) and maps it to a corresponding output Y (e.g., a class label), which can be defined as a function $M : X \Rightarrow Y$. Specifically, an inference phase takes three steps. First, the model M passes the input X through all the operators and computes $Z(X)$ which is known as the *logits* [35]. Second, M utilizes the *softmax* function to normalize the values of $Z(X)$ and computes an output vector $V_i(X) = softmax(Z(X))$, where each element in $V_i(X)$ represents the probability of the input X belonging to class i while holding $V_i(X) \in (0,1)$ and $\sum V_i(X) = 1$. Last, M computes and assigns the final class label $Y = argmax_i V_i(X)$ to the input X.

DNN operators. Operators are the key building blocks of a model where linear operators are proved to be the main computational bottleneck in inferences. For instance, for a classic VGG-19 inference on a single CPU thread, about 98% of the computation is spent on the linear operators [83].

Figure 1 shows two typical forms of linear operators: *convolution* and *fully-connected*. During the convolution computation, the convolution kernel (with its parameters) is placed at different locations of the input data. At each location, a sum of element-wise product is computed between the kernel and corresponding input values within the kernel window. The fully-connected operator conducts a weighted sum (i.e., a dot product) between an input and a flatten parameter vector which is owned by the fully-connected operator.

Nonlinear operators typically use *activation* functions, such as Rectified Linear Unit (*ReLU*): $f(x) = max\{0,x\}$ and *Sig-*

moid: $f(x) = 1/(1+e^{-x})$ to approximate complex transformations [3, 95], and use *pooling* functions, such as the *max-pool* and *mean-pool* to reduce the dimension of intermediate computation results produced in inference computations [24].

Associativity. Suppose we have an input X and a scalar μ, a DNN operator F is associative if

$$(\mu^{-1} * \mu)F(X) = \mu^{-1}F(\mu X) \quad (1)$$

Typical linear operators (e.g., *convolution*, *fully-connected*) all have the associativity property because these operators conduct linear transformations on an input data. Take the most commonly used, computation-expensive convolution operator [76] as an example. As shown in Figure 1, if we multiply each element in the convolution kernel by 2^{-1} (i.e., $\mu = 2^{-1}$), we will get the output $2^{-1}R$ while it always holds $(2 * 2^{-1}) * R = 2 * (2^{-1}R)$. Hence, the *convolution* operator is associative. This is applicable to the *fully-connected* operator as well.

Since nonlinear operators conduct nonlinear transformations on an input data, nonlinear operators usually do not have an associativity property (e.g., *Sigmoid*). However, under specific constraints, some nonlinear operators can also be associative. Specifically, when $\mu > 0$, the *ReLU* operator is associative because μ will not change the positive or negative of the input X, such that $\mu^{-1} max\{0, \mu X\}$ will get the correct output. The same constraint is applicable to the *max-pool* operator as well. Note that the *mean-pool* operator naturally has the associativity property because it actually conducts linear transformation (i.e., averaging) on an input data.

Operators that satisfy Equation 1 also meets a variant of the associativity property. Given that

$$\begin{cases} F(X_1) = Y_1, \\ F(X_2) = Y_2. \end{cases}$$

we have

$$F(\mu_1 X_1 + \mu_2 X_2) = \mu_1 Y_1 + \mu_2 Y_2 \quad (2)$$

SOTER leverages Equation (1) and Equation (2) as the key weapon to automatically partition general models without needing to identify security-sensitive operators in a handcrafted manner, which is error-prone. We illustrate our detailed designs in §4.

2.2 Intel SGX

Intel Software Guard eXtension (SGX) [12] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called enclave, where data and code execution can not be seen or tampered with from outside. Code outside enclaves can enter an enclave by `ECalls`, and SGX uses remote attestations [12] to prove that a particular piece of code is running in an enclave on a genuine SGX-enabled CPU. SGX provides a trustworthy random source (`sgx_read_rand`), which invokes a pseudo-random generator

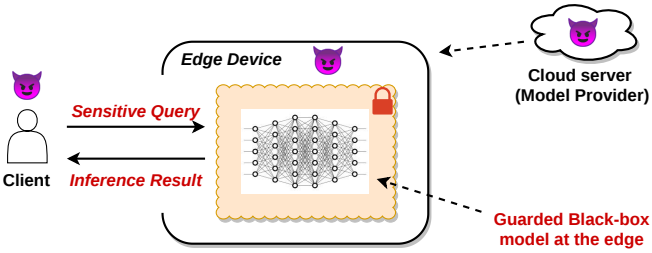


Figure 2: The client-side model inference scenario considered by SOTER.

through the RDRAND CPU instruction seeded by on-chip entropy sources [12]. Previous studies show that this random source complies with security and cryptographic standards and can not be seen or tampered with from outside enclave [7].

Recent work leverages SGX to guard black-box model inference at the edge. MLcapsule [33] encapsulates individual model layers and run inference computations purely inside the SGX. Such a modular approach makes it east to combine layers and form large networks but fails to provide fast inference speed owing to the computational bottleneck of the CPU.

Same as SOTER, eNNclave [70] and DeepEnclave [32] also take a partition approach to utilize a co-located GPU for low-latency inference, which is the dominant approach for secure inference at the edge. However, these two systems require manual effort to identify sensitive layers and partition the insensitive part from enclave to GPU. Such a handcrafted approach is only secure for specific models with lucid sensitivity boundaries among layers. Moreover, exiting partition-based approaches do not handle the inherent integrity threat after model partitioning, exposing a large attack surface to the adversary outside SGX enclave.

3 Overview

3.1 System setup

We consider a client-side inference scenario shown in Figure 2. There are two parties in the system: the client and the model provider (or server). The client owns sensitive inference queries for a specific AI task. The server owns an appropriate model that has well-designed model architecture and well-trained parameters for the task. The server offloads its model to an SGX-equipped edge device. The offloaded model conducts client-side inference by taking client's sensitive queries, then sends the inference results back to the client.

3.2 Threat model and privacy goals

Similar to other client-side secure inference systems, namely the MLcapsule [33], eNNclave [70] and DeepEnclave [32], SOTER adopts a semi-honest model, where both the client and

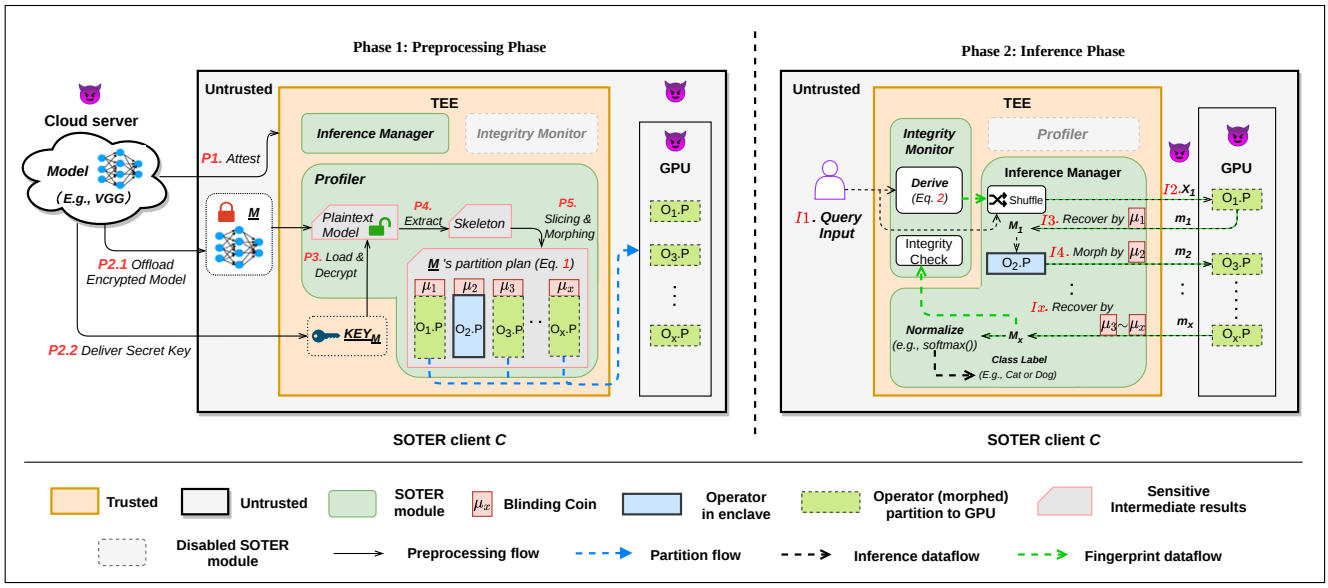


Figure 3: The architecture of SOTER in a modular manner. SOTER's secure inference protocol has three modules running in the enclave (shielded in green). An adversary can observe and modify parameters of partitioned operators in the co-located GPU.

server have a bounded computational capability [96, 99], and try to learn information about the other party's private data from the messages it receives. That is, the client C and server S will follow the inference protocol but C wants to learn S's proprietary model architecture and parameters, and S attempts to learn C's input data. Since the two parties have conflicting interests, we exclude collusion attacks here.

Different from prior work that assumes the offloaded model is built on well-known DNN architectures such as ResNet [82] and VGG [10,67], SOTER protects both model parameters and architecture (number of layers, linkages between layers, etc) because recent advances in *Neural Architecture Search* (NAS) has demonstrated that model architecture has a significant impact on achieving high inference accuracy, making a well-designed architecture proprietary to get protected [4, 9, 19].

Note that, SOTER does not hide information that is revealed by the results of inference queries and focuses just on protecting clients' data and the offloaded model during the inference process. This assumption stays in line with the threat model of previous work, including both client-side and cloud-side secure inference systems [33, 40, 55, 70]. Protecting against such attacks that exploit the leakage of inference results is a complementary problem to that solved by SOTER.

Indeed, many attacks have been successfully carried out even against systems that completely hide the model with black-box accesses [61, 90]. However, existing popular mitigations for these attacks, such as query auditing [39] and differential privacy [18], can be combined with SOTER's protocol with sufficient engineering effort. We give in-detail illustration of these attacks and mitigations in §7.

SGX's threat model. SOTER has the same threat model for SGX as typical SGX-based systems [38, 74, 86]. We trust the hardware and firmware of Intel SGX, which ensures that code and data in enclave can not be seen or tampered with from outside. Besides, we trust that SGX's remote attestation service can identify genuine SGX devices from fake ones (e.g., emulated with QEMU). Same as eNNclave [70] and other SGX-based secure inference systems [32, 33], we do not intend to address the vulnerabilities of side-channel attacks in this paper, but we still discuss the potential vulnerabilities and mitigations in §7.2.

We also consider a third-party attacker that can hack into the client device, observe and manipulate any components outside enclave (e.g., parameters in GPU, intermediate results transmitted between SGX and GPU, etc) to cause integrity breaches during the inference process. Such attack widely exists in an edge computing ecology [29, 42, 49]. For instance, an attacker can hack into an autopilot system, perturb the obstacle prediction to produce wrong navigation instructions [69].

3.3 System overview

SOTER is a secure inference system running at the client side. Different from a cloud-side system (e.g., Delphi [55]) that aims to protect the client's sensitive input data, in SOTER, the client's data is never transmitted to the untrusted cloud, hence SOTER naturally protects the client's data. Instead, SOTER focuses on protecting the offloaded model at the client-side. A client runs SOTER's protocols in a trusted module (i.e., the SGX enclave) and all system components outside enclave can behave arbitrarily.

SOTER's architecture. Figure 3 shows the architecture of a SOTER client. The client is equipped with an attested SGX enclave running only SOTER's inference protocols. Each SOTER

client runs the MTR protocol in three trusted modules (in green) within enclave (in orange). The *profiler module* takes an offloaded encrypted model and its corresponding keys as input to extract the model skeleton (i.e., architecture), and automatically makes an optimal partition plan. The *inference manager module* takes the client's query input (i.e., data for inference) and manages inference computations according to the partition plan. The *integrity monitor module* dynamically derives fingerprint data flows with *oblivious fingerprints* that are indistinguishable to the client's inference data flows from an adversary's view outside enclave.

SOTER's two-phase design. At a high level, SOTER's protocol for secure inference at the client-side consists of two phases: an offline *preprocessing phase*, and an online *inference phase*. Specifically, the offline preprocessing phase is independent of the client's query input which changes regularly. We assume the offloaded model from the server is static, if the model changes, then both parties should re-run SOTER's preprocessing phase. After preprocessing, during the online inference phase, the client sends query input to get the eventual inference result by passing through SOTER's integrity monitor module and inference manager module. Note that, SOTER's two-phase design is most suitable for applications which could reserve sufficient preprocessing resources (see **Remark I**).

SOTER's workflow. In Figure 3, we show how SOTER leverages the general associativity property of DNN operators to automatically partition a model without identifying the sensitive (or insensitive) parts, which makes SOTER unique to provide strong confidentiality guarantees to general models. We also introduces how SOTER guards the entire inference process in two phases.

In the preprocessing phase: (**P1**) The server S attests to the SOTER client C to ensure legitimate SOTER's code is running on top of a genuine SGX-enabled hardware. (**P2.1**&**P2.2**) If the attestation fails, then the preprocessing phase aborts; Otherwise, after a successful attestation, S builds up a secure communication channel with C, then S offloads its encrypted model M to the untrusted memory outside enclave, and S also offloads the corresponding decryption keys KEY_M to C's enclave. (**P3**) SOTER's trusted profiler module loads the encrypted model M in a layer-wise manner and decrypts M with KEY_M. Note that, for the latest version of SGX with sufficient memory support [75], we could directly load the offloaded model to enclave. (**P4**) With a plaintext model, SOTER extracts the model skeleton to capture the architecture information including the operator types, the number of layers, and the linkages between layers. (**P5**) With the architecture information, SOTER statically filters out all *associative* operators that meets Equation 1, and then invokes SOTER's MTR protocol to securely partition part of model operators to GPU for faster inference computing.

Specifically, in **P5**, SOTER's MTR protocol runs as follows: With a given security parameter θ, SOTER randomly selects θ% associative operators, and runs SGX's pseudo-random number generator (PRNG) to produce (1) *fingerprints* of the selected operators for integrity checking (used in the inference phase), and (2) random positive scalars for all operators. These numbers serve as the *blinding coins* to hide the operators' parameters [55] and are always kept secrete in enclave. Owing to the associativity property (Equation 1), SOTER morphs every selected associative operator by multiplying each element in the operator's parameter matrices with a blinding coin (the computation result can be recovered by using the reciprocal coin), and then partitions the selected operators with morphed parameters to GPU. In Figure 3, the selected operators (in green) are morphed with corresponding coins, e.g., the parameter of the third operator $O_3.P$ is morphed with a coin μ_3 and partitioned to GPU, while the second operator (in blue) is kept within enclave. In §5.1.2, we illustrate our choice of θ by considering the *transferability* of a partitioned model, which is a widely used metric to evaluate a black-box model's confidentiality under adversarial attacks [26, 52, 84, 99].

Note that, we choose to run the preprocessing procedures (**P1**∼**P5**) at the client-side because a client C might select his own θ according to his GPU capability, e.g., choose a smaller θ if the GPU has limited usable memory, even if SOTER has chosen an appropriate θ (see §5.1.2) for confidentiality.

In the inference phase, there are two types of data flows: the inference flow (black dotted line) and the fingerprint data flow (green dotted line). The life-cycle for the inference dataflow is: (**I1**) The client C sends a query input (i.e., data for inference) to the trusted inference manager module and shuffles with the fingerprint input (introduced below). (**I2**) The inference manager forwards C's input X_1 to the GPU, and the first *morphed* operator $O_1.P$ takes X_1 as input and computes the output m_1. (**I3**) Since the next operator $O_2.P$ is expected to run within enclave according to the plan made in the preprocessing phase, the inference manager takes the computation result of previous partitioned operators (i.e., m_1) as the input, recovers m_1 with all previous partitioned operators' blinding coins since the last kernel switch (in Figure 3's circumstance, SOTER recovers with only $O_1.P$'s blinding coin μ_1), and gets the recovered result M_1. (**I4**) After that, the second operator $O_2.P$ computes with M_1 and gets an intermediate result (IR), then morphs IR with the $O_2.P$'s blinding coin μ_2 and forwards the result m_2 to the partitioned operator $O_3.P$ in GPU.

The above procedures (**I2**∼**I4**) repeat until all partitioned operators in GPU are computed. In (**Ix**), with the last computation result m_x from $O_x.P$, SOTER recovers the real computation result by using all previous partitioned operators' blinding coins since the last kernel switch (same as the recover step in **I3**). In this case, SOTER recovers with $\mu_2 \sim \mu_x$ and gets the real inference result M_x. Then, SOTER runs a standard DNN normalization function (e.g., softmax()) on M_x to get the class label and returns the label to the client C.

The fingerprint data flow is used to detect integrity breaches of partitioned operators outside enclave. Specifically, with a

Table 1: SOTER's protocol variables.

Variable	Description
$\Theta(default = 0.8)$	Portion of associative operators partitioned to GPU.
O_g	The set of partitioned operators in GPU.
O_e	The set of operators maintained in enclave.
$O(O_g \cup O_e)$	The whole set of model M's operators.
$Para(O_i)$	Parameters of operator O_i.
FP_{O_i}	Fingeprint of operator O_i.
μ_i	Blinding coin of operator O_i.
$K(default = 10)$	The initial number of fingerprints for each $O_i \in O_g$.
$W(default = 2)$	Top-W operators are always maintained in enclave.

set of partitioned operators' fingerprints produced in the preprocessing phase, within the inference data flow, SOTER's trusted integrity monitor module dynamically derives oblivious fingerprints by utilizing the general associativity of Equation 2, injects these fingerprints into the inference data flow to check whether the parameters of partitioned operators (or the IRs) have been modified by the adversary outside enclave. We prove in §5.2 that the adversary outside enclave can not distinguish the fingerprint flow from the inference data flow, preventing the adversary from bypassing the integrity checking.

Remark I (SOTER's targeting applications). Same as other secure inference systems that involve two phases (Delphi [55], Gazelle [40]) (although they are designed for cloud-based secure inference), SOTER is best suited for applications whose inference is latency-sensitive, but is usually not performed frequently enough to take up all computational resources needed for preprocessing. Examples of such applications include speech-to-text systems like [27] and image classification systems like Google Lens [28].

4 Protocol Description

This section describes SOTER's MTR protocol for partitioning general DNN models with confidentiality and integrity guarantees. At a high level, MTR utilizes the general associativity property of many DNN operators to automatically profile the entire model (with Equation 1), randomly selects θ% ready-to-partition operators to hide the model architecture, and then morphs these operators' parameters with hidden *blinding coins* in enclave to hide the plaintext data (§4.1).

Besides, to tackle the inherent integrity threat of inference manipulations from a third-party attacker (described in §3.2), stemming from the same observation of the general associativity property, MTR dynamically derives *oblivious fingerprints* (with Equation 2) and uses fingerprints in enclave to check operation integrity in GPU in a *challenge-proof* manner (§4.2).

We now give the formal definition of a client-side partition-based secure inference protocol as follows.

Definition I. *An inference protocol ψ running at client C holds an offloaded model M with architecture α and parameters $(M_1, ..., M_l)$, and the client C holds input queries Q to run inference on M. ψ is a highly-desirable secure inference protocol at the client side if it satisfies the following rules:*

- ***Security:***

- ***Corrupted server.*** *We require that a corrupted, semi-honest server does not learn anything about the input queries Q of the client C.*

- ***Corrupted client.*** *We require that a corrupted, semi-honest client C does not learn anything about either the model M's architecture α or the plaintext parameters $(M_1, ..., M_l)$. Formally, with a partitioned model M_p under the view of C, we require that (1) ψ should guarantee a low probability to derive the true parameters $(M_1, ..., M_l)$, and (2) ψ should guarantee a low transferability of M_p, which is defined as the ratio of the correctly computed inference queries that the partitioned model M_p can mimic the original model M (see §5.1.2).*

- ***Hijacked client.*** *Given that a client C might be hijacked by a third-party attacker, who can observe and manipulate the inference process to harm the service, we require that ψ has to quickly detect integrity breaches outside enclave with overwhelming high probability and zero false positives.*

- ***Performance:***

- ***Low latency.*** *We require that without sacrificing security, ψ should partition as many computations as possible to a co-located accelerator (e.g., GPU) for speeding up inference, given that client-side AI applications always have stringent latency requirements.*

- ***High accuracy.*** *Given a partitioned model M_p, on every input query Q from client C, we require that the output $M_p(Q)$ returned by ψ should achieve the same inference result as the original model $M(Q)$ without any accuracy loss.*

4.1 Morph Then Recover (MTR) protocol

The MTR protocol is divided into two stages: a *morphing* stage in the preprocessing phase (handled by the profiler module), and a *recover* stage in the inference phase (handled by the inference manager module). In the morphing stage, SOTER first makes *blinding coins* for every operators (including both associative and non-associative operators), and then makes a partition plan with a given security parameter θ. Then, in the recover stage, SOTER runs inference across GPU and enclave, and recovers inference results in enclave when needed. Algorithm 1 and 2 show the MTR protocol.

Stage 1-1: Morphing with blinding coins. SOTER assigns every operator with a randomly generated scalar, which serves as the blinding coin (in short, coin) to hide the plaintext value of parameters. Given a ready-to-partition operator O_i (the partition plan is described in **Stage 1-2**), SOTER morphs $Para(O_i)$ by multiplying each element in $Para(O_i)$ with the corresponding coin μ_i. In our implementation, we use the trustworthy random source (`sgx_read_rand`) in SGX to generate coins and and assign coins to every operator in model M (Line 1∼6 in Algorithm 1).

Algorithm 1: MTR protocol at client c (offline) (§4.1).

```
▽ Preprocessing Phase (within enclave).
1  Function make_coin(O) do
2      μ ← ∅;
3      foreach operator op in O do
4          index ← O.index(op); coin ← ∅;
5          while coin = ∅ || coin = sgx_read_rand() < 0 do
6              μ ← coin;
   ▽ Partition Phase (within enclave).
7  Function partition(O) do
8      foreach operator op in O do
9          op.partition ← False; op.index ← O.index(op);
10         if op.index > W then
11             if 0 < normalize(sgx_read_rand()) < Θ then
12                 op.partition ← True;
13             if op is associative & op.partition then
14                 foreach element e in para(op) do
15                     e ← μ_index × e ;            // Morph
16                 O_g.add(op) ;          // GPU operators
17             else
18                 O_e.add(op) ;          // TEE operators
19     O_g.copyTo ("cuda") ;          // Partition to GPU
```

The blinding coin design protects model confidentiality from two aspects. First, by hiding coins within enclave, we intuitively ensure that an adversary outside enclave cannot learn the real parameters before being morphed in enclave.

Second, we assign every operator (rather than every ready-to-partition operator) with coin to avoid potential information leakage during kernel switches, i.e., when intermediate results (IRs) are transmitted between enclave and GPU. Consider a case in Figure 3 where O_1 (in GPU) is a linear operator and O_2 (in enclave) is a non-linear *ReLU* operator. If we only morph O_1 with μ_1 and do not assign a coin to O_2, then, with a client input X, O_1 outputs $y_1 = X(\mu_1 * O_1.P)$ and O_2 outputs $y_2 = ReLU(y_1/\mu_1) = ReLU(X * O_1.P)$. However, since the *ReLU* operator just filters out negative values in parameter matrices and does not transform a scalar, an adversary who observes both y_1 and y_2 will directly infer the value of coin μ_1, violating the confidentiality of the partitioned O_1 (concretely, $Para(O_i)$). To tackle this problem, we also assign coin μ_2 to O_2 and O_2 will output $y_2 = \mu_2 * ReLU(X * O_1.P)$. An adversary can observe $\mu_1 * \mu_2$ but cannot infer either μ_1 or μ_2.

Stage 1-2: Partitioning with hidden architecture. With all coins prepared, given a security parameter θ, we automatically partition a portion of associative operators (that are morphed) to GPU for inference acceleration. The *partition* function in line 7∼19 of Algorithm 1 shows the pseudo code.

Specifically, in the preprocessing phase, SOTER iterates every operator in model M and randomly selects a portion of operators and adds them to the partition set O_g. When meets an associate operator, SOTER uses the trustworthy source of SGX to generate a normalized random number, if the normalized number is within the range of $0 \sim \theta$, then the operator is added to the partition set O_g. For instance, with $\theta = 0.8$, an

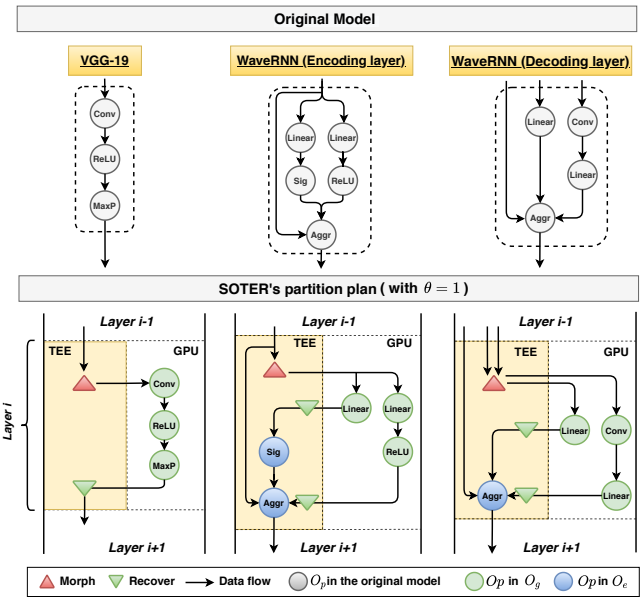


Figure 4: Examples of running SOTER's MTR protocol to partition two typical DNN models.

associative operator would have an 80% chance to be partitioned to GPU. After all operators are iterated, all operators in O_g are partitioned to GPU for inference acceleration.

Note that, SOTER always keeps top-W (W=2 by default) operators in enclave even if these operators are associative (Line 10). This design choice is made to ensure the input to the first several partitioned operators in GPU (e.g., X_1 in Figure 3) are always unknown to the adversary, such that we can stealthily check the integrity of all partitioned operators by injecting fingerprint challenges (more details in §4.2).

Figure 4 depicts an example of running the MTR protocol to automatically partition general neural networks. We use a VGG-19 model and a WaveRNN model here: VGG-19 is a popular CNN model for image classification tasks [10]; WaveRNN is a state-of-the-art RNN model widely adopted by the industry for efficient speech-to-text transformation [41]. To illustrate our protocol in a lucid manner, in this example we choose $\theta = 1$, which means that all associative operators will be partitioned to GPU for acceleration.

In the above example, for VGG-19, we can partition most of its operators to GPU because VGG-19 is a typical layered model composed mostly by associative operators including *convolution*, *ReLU* and *max-pool*, given that the last two non-linear operators can also meet the associativity property with a positive coin (§2.1). Besides, most non-associative operators in VGG-19 such as *BatchNormal2d* and *DropOut* are automatically de-activated during the inference process to avoid non-determinism [73]; For WaveRNN, we give partition plans for its encoding (decoding) layer as WaveRNN is built by iteratively stacking these two layers and other scattered layers. Different from VGG-19, as a typical RNN model, WaveRNN has more non-associative operators (e.g., *Sigmoid*) that cannot

Algorithm 2: MTR protocol at client c (online) (§4.1).

```
   ▽ Inference Phase (cross enclave & GPU).
1  Function secure_inference() do
       ▽ I1.
2      every client input X do
3          X.copyTo("enclave");
4          foreach index i ∈ normalize(W) do
5              X ← O_i(X) ;                    // Top-W Inference
6          start ← next op in O_g; end ← next op in O_e;
7          while start < |O| do
               ▽ I2.
8              X.copyTo("cuda");
9              foreach index i ∈ range(start, end) do
10                 X ← O_i(X) ;                // GPU Inference
               ▽ I3.
11             X.copyTo ("enclave");
12             X ← ∏_{start}^{end-1} μ_i^{-1} × X ;   // Recover
13             start ← end;
14             end ← start from O_end find the next op in O_g;
15             foreach index i ∈ range(start, end − 1) do
16                 X ← O_i(X) ;                // TEE Inference
               ▽ I4.
17             X ← ∏_{start}^{end-1} μ_i × X ;        // Morph
18         X ← normalize(X);
19         return Top_1 (X) ;                  // Final result
```

be partitioned to GPU for acceleration.

Stage 2: Guarded black-box inference. Next we present the end-to-end black-box inference process given that we have prepared coins and partitioned a portion of operators to GPU.

As shown in Algorithm 2, SOTER first computes the reserved top-W operators in enclave, and then iterates **I2**∼**I4** as follows. **I2**: SOTER finds the longest length of consecutively partitioned operators in O_g and computes the inference result by forwarding the input through these operators (line 6∼11). **I3**: Then SOTER copies the result back to enclave, recovers the real inference result by multiplying the reciprocal coins of all operators in last step (line 13), and then forwards the result to operators maintained in enclave (before the next copy to GPU). **I4**: SOTER morphs the forwarding result with all coins of enclave operators in last step (line 18), and then copies back to GPU. This procedure terminates when all partitioned operators in GPU are iterated. Then SOTER normalizes the final result and returns the result to the client.

4.2 Integrity checking with fingerprints

Obliviousness requirement. To detect integrity breaches outside enclave (as described in §3.2), a straw man approach could be using a fingerprint (with ground-truth input/output pair) to *challenge* the partitioned operator in GPU, recompute the fingerprint input to provide a *proof*, and report a crime if the proof is different from the expected output.

However, using fingerprints to detect integrity breaches outside enclave needs to be oblivious: since the adversary can continuously monitor the inference process and observe every intermediate result (IR) transmitted from enclave to GPU, if using only a fixed set of fingerprints, the adversary can easily distinguish those fixed fingerprint challenges among all IRs, thereby bypassing our detection by returning correct proofs.

The timeliness-obliviousness dilemma. Unfortunately, trivially using different fingerprints for integrity checking would bring a timeliness-obliviousness dilemma.

Specifically, if we generate different fingerprint input (serve as *challenges*) and pre-compute expected output (serve as expected *proofs*) at runtime, that would cause late detection because we have to run slow CPU pre-computation (up to 30x slower inference speed for linear operation) in enclave for each partitioned operator and for each query from client; On the other hand, if we use a fixed set of fingerprints for challenging the integrity, we could timely detect breaches because we no longer have to make new fingerprints with CPU inference, however, the detection would be easily bypassed by the adversary because all fingerprints are now distinguishable.

Insight and approach. Stemming from the same observation of the general associativity property of many DNN operators, we tackle this challenge by efficiently generating oblivious fingerprints and minimizing redundant CPU pre-computation. At a high-level, we take two steps by first making a set of *cornerstone fingerprints* for each partitioned operator, and then dynamically deriving oblivious fingerprints from these cornerstone fingerprints with linear cost at runtime based on the associativity property.

Step 1: Preparing cornerstone fingerprints. During the preprocessing phase (line 18∼19 of Algorithm 1), SOTER prepares K (by default K=10) cornerstone fingerprints for each partitioned operator. A fingerprint is a two-element tuple: $\{input, output\}$, where the input is a randomly generated matrix and the output is pre-computed by forwarding the input through a ready-to-partition operator. The whole preparing procedure is running within enclave, such that the correctness of each cornerstone fingerprint can be guaranteed.

Step 2: Efficiently deriving new fingerprints in TEE. Now with K cornerstone fingerprints for each partitioned operator, SOTER efficiently derives oblivious fingerprints at runtime by leveraging the general associativity property. Specifically, with a fixed set of cornerstone fingerprints $FP_O[K]$ for each partitioned operator O, we randomly selects T fingerprints and generates T random coefficients with SGX's trustworthy random number generator. Then, by applying the associativity property (Equation 2), we generate a new fingerprint FP_{New} as follows: the input for FP_{New} is $\sum_{i=1}^{T} T_i * FP_O[i].input$, i.e., we multiply each selected cornerstone fingerprint's input with a corresponding coefficient and add them all as the new input; For the output, since the associativity property implies that the corresponding output for a group of associated operators has the same transformation as the input, thus we can directly get the expected output as $\sum_{i=1}^{T} T_i * FP_O[i].output$ without conducting slow CPU inference for a given new input.

Step 3: Challenge-proof fingerprint issuing. To detect any integrity breaches on partitioned operators with overwhelming high probability (see **Remark II**), we issue a ***challenge*** with probability p whenever a kernel switch (an IR transmitted from enclave to GPU) happens. In more detail, when an IR (produced by client's query) is sent to a partitioned operator O, we challenge the integrity of O by sending with a new $FP_O.input$ (produced in **step 2**), and compare the returned ***proof*** with the expected $FP_O.output$. Any mismatch reveals an integrity breach of partitioned operator O.

Remark II (Detection probability and performance impact). The challange probability p affects both the detection rate and performance: adding p will increase the detection probability and timeliness, but comes at a runtime overhead to GPU. We prove (and evaluate) that SOTER can achieve overwhelming high detection probability (§5.2) and reasonable performance overhead (§6) with a proper choice of p.

5 Analysis of SOTER Security Properties

In this section, we present the analytical and experimental evaluation on the security of SOTER. We first demonstrate that our MTR protocol provides sufficient query confidentiality and model confidentiality during the inference process. Then, we analyze the integrity under random perturbation attacks, and show that our solution can handle such cases gracefully.

5.1 Confidentiality

We analyze the confidentiality by considering the following adaptive attackers with respect to our threat model (§3.2).

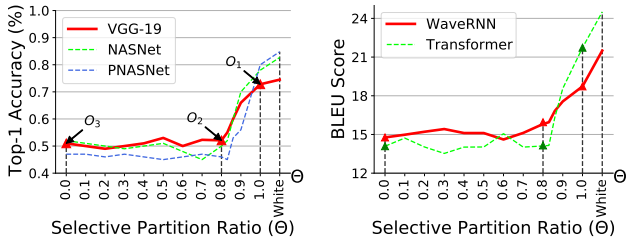
5.1.1 Corrupted Server

Attacker 1. We consider a malicious server S who wants to abuse client's input queries X. Such vulnerability widely exists in AI-driven applications. For instance, in face recognition, S might maliciously collect client's face images to strengthen (train) its own model [22].

Intuitively, such an attacker is naturally warded off by the design of SOTER. This is because all communication between client and server happens before the queries being observed by the server. As the client makes inference offline and the results were generated within the trusted enclave (shown in Figure 3), we ensure that the input queries will never get leaked and a malicious server cannot break the query confidentiality.

5.1.2 Corrupted Client

Attacker 2. We consider a semi-honest (i.e., honest but curious) client who follows our MTR protocol coded within SGX but wants to train a *substitute model* with several observations of partitioned model architecture and parameters outside enclave. Such adversary could be a business competitor whose purpose is to steal the offloaded model, in order to reduce the competitive advantages of the model owner.



(a) CNN models (b) NLP models

Figure 5: SOTER's θ-Transferability analysis of prevalent DNN models on the adversarial substitute model $SM(\theta)$.

We emphasize two assumptions ahead of the attack analysis here. First, we assume the adversary has a *bounded computational capability* as in [61, 89], and does not know the whole training dataset. Otherwise, it is totally unnecessary for the adversary to "steal" the model. This assumption stays in line with existing partition-based inference systems to evaluate the confidentiality guarantee of a model's partition plan [89, 99].

Second, we do not consider any black-box attacks here and we assume client's queries are *benign*. While a corrupted client might use SOTER as a black-box oracle to extract the model or even the training dataset [50, 61, 63, 90], SOTER does not aim to protect against such black-box attacks, which stays in line with the threat model of either cloud-side or client-side state-of-the-art black-box inference systems [33, 40, 55, 70]. SOTER focuses just on protecting the offloaded model's well-designed architecture and well-trained parameters during the inference process (more discussion in §7.2).

Evaluation Metric. Due to the heuristic nature (weak explainability) of DNN models [20], it is hard to give a formal theoretical analysis of the confidentiality guarantee. Instead, same as prior work [89, 99], we give an experimental security analysis to evaluate the model confidentiality of our MTR protocol with the *transferability* metric.

Transferability means the portion of functionalities that a substitute model SM (trained by the adversary) can mimic the original offloaded model M [89]. A higher transferability indicates that the SM trained by the adversary can mimic a larger portion of functionalities (i.e., more confidentiality loss), while a lower transferability reveals that a smaller portion of functionalities have been stolen (i.e., less confidentiality loss). Concretely, for CNN models, we use the *Top-1* accuracy as the transferability metric and we use the *BLEU score* for the RNN models, both of which are well-known AI metrics to measure the quality of a trained model [65].

Substitute training setup. We experimented the adversarial substitute training (*Attack 2*) with five prevalent DNN models, pre-trained them with CIFAR-10 dataset [43] (for CNN) and WMT16 EN-DE dataset [71] (for RNN and transformer), respectively. With the bounded computational capability assumption, to stay in line with [61, 99], we isolated 90% of

training data (e.g., 45,000 images in CIFAR-10), and the remaining 10% of training samples are used (and augmented with [61]) by the adversary.

Following state-of-the-art substitute training setup [89], to construct the substitute model, we adopted VGG-13 as the backbone network for CNN models, and we used transformer (with encoder-decoder architecture) as the default substitute model for NLP models. With the substitute model, for operators having the same size as the observed target operators (with morphed parameters) outside enclave, we simulated the adversary by fitting these observed target operators into the substitute models; for other operators, we initialized all parameters using a truncated normal distribution with std of 0.02. Same as [89], we used Adam to train the substitute model with 0.0001 learning rate. By training with one Nvidia 2080TI GPU, the CNN models gradually converged between 80∼150 epochs while NLP models converged within 10 epochs.

θ-Transferability analysis. We now analyze the model confidentiality of our MTR protocol. In particular, we give experimental results on the effects that the selective partition ratio θ could have on the quality of the adversarial substitute model *SM* trained by the adversary. With a given θ, we denote the corresponding substitute model as SM_θ.

To give analysis in a lucid manner, we first analyze the transferability of SM_θ when the target model is a typical CNN model VGG-19, and then we give analysis on other prevalent DNN models. Specifically, for VGG-19, we simulated an adversary who trains different *SM*s based on the following partial views outside enclave, as shown in Figure 5a.

- *White-box model.* The red line (VGG-19) can achieve the ground-truth Top-1 accuracy (74.5%) when the X-axis is "*White*", i.e., a scenario where the entire model was placed in GPU without activating our MTR protocol (i.e., SM_{White}). The ground-truth accuracy result is intuitive due to the white-box model outside enclave being the same as the original model.
- *Fully Black-box model.* When the X-axis = 0, which means no operators were partitioned to GPU (i.e., fully black-boxed in enclave), the adversarial substitute model SM_0 achieved 54.3% accuracy, showing a low transferability of the target model *M*. This also indicates that even with a fully black-box model, the adversary can still train a substitute model from a black model with a partial dataset through queries.
- SOTER *Morphed model.* When the selective partition ratio θ downgrades from 1 to 0, we observed a significant accuracy (transferability) drop from 71.8% to 54.3%, which shows the effectiveness of using blinding coins to hide parameters. Besides, we emphasize three key points (**O1**∼**O3** in Figure 5a) from our experiments.

First (**O1**), when the X-axis = 1, SM_1's accuracy decreases ∼2.7% because in that case the adversary can train SM_1 with the knowledge of (1) the partial architecture of model *M* (since operators used in VGG-19 are mostly associate, the adversary will get most of the operators' information when θ = 1), and (2) morphed parameters of all associative operators partitioned to GPU. With these knowledge, the adversary could achieve a resultant substitute model SM_1 with a relative high accuracy (i.e., high transferability).

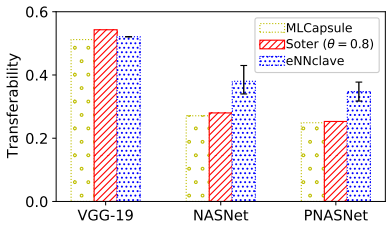


Figure 6: Comparison of transferability between SOTER and other baseline systems. A lower transferability (Y-Axis) indicates a stronger confidentiality guarantee on resisting adversarial substitute training (***Attacker 2***) on that model (X-Axis).

Second (**O2**), when SOTER chooses θ = 0.8, we observed a significant accuracy drop to 54.7%. This is the scenario where SOTER partitions any associate operators to GPU with 0.8 probability. In that case, we hide the architecture information of the model, and the adversary can only train $SM_{0.8}$ with fragmented operators' information and morphed parameters of only a fraction of partitioned associative operators. As a result, the adversary could only train a low-accuracy model $SM_{0.8}$, which can not substitute the original high-accuracy model *M* because of the low transferability.

Last (**O3**), when the selective partition ratio was within the range of 0∼0.8, we observed that the accuracy was almost unchanged. This is because, even with a smaller θ, which means less associative operators were partitioned to GPU, the difficulty for the adversary to train a substitute model SM_θ remained almost unchanged, i.e., the transferability of a substitute model SM_θ ($\theta \in (0, 0.8)$) remained almost unchanged.

Based on these results, we choose θ = 0.8 in SOTER by default, which obtains the maximum performance benefit (as most associate operators would be partitioned to GPU for fast processing) when achieving a comparable confidentiality guarantee as fully black-boxed models (i.e., θ = 0). We also extended the transferability analysis to NASNet, PNASNet, WaveRNN and Transformer, shown in Figure 5a and Figure 5b. The similar downgrading trend and θ-*Transferability* relationship revealed that hiding with 80% morphed information could be enough to guarantee sufficient model confidentiality (with a low transferability).

Baseline comparisons. MLcapsule [33] is a pure TEE-shielded inference system, which prohibitively incurs high inference latency but achieves the strongest model confidentiality because all operators are shielded within trusted enclave (corresponds to the θ = 0 scenario in SOTER). As shown in Figure 6, MLcapsule indeed has the lowest transferability, but is similar to SOTER (θ = 0.8). This comparison verifies our analysis (**O3**) and supports our parameter choice (θ = 0.8).

Another baseline eNNclave [70] takes a handcrafted approach to manually identify insensitive layers, fine-tune these layers' parameters and directly partition them to run in GPU

(see more details in §7.1), which requires substantial expertise and usually causes accuracy drops [70]. As eNNclave does not mention any strategies in its paper for partitioning general models other than VGG-series, to fairly compare with it, we randomly partitioned complex models and recorded transferability fluctuations as error bars, shown in Figure 6. eNNclave only provided sufficient confidentiality (low transferability) to VGG model, because simple models always have lucid sensitivity boundaries between layers [37, 81, 92]. Thus protecting several specific DNN layers [70] could be enough to achieve strong confidentiality in simple models, which is not applicable to general complex models, proved by the much higher transferability than SOTER and MLcapsule. This comparison also verifies that our protocol of using blinding coins to morph parameters is effective.

Overall, we analyzed the confidentiality of MTR - our parameter-morphed client-side inference protocol, and confirmed the effectiveness of using blinding coins to morph parameters (see Figure 5). Compared to baselines, we (at the first time) achieve both efficiency (GPU-accelerated) and privacy (low-transferability) for general DNNs (see Figure 6), making SOTER unique to serve both performance- and security-critical applications at the edge [8, 48, 60].

Attacker 3. We consider a semi-honest client who continuously monitors the inference process and collects a set of morphed operators outside enclave, aiming to derive the scalars or true parameters.

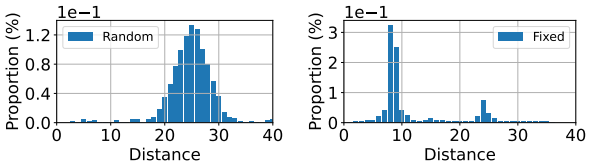
We prove our security by demonstrating the theoretical low probability of successfully launching such an attack. Specifically, even with several observations of a set of morphed parameters of a specific operator O, that is, even the adversary observes N morphed parameters ($mp_0 \sim mp_N$) hidden with scalars ($\mu_o \sim \mu_N$), the adversary still has a low probability to derive the true $Para(O)$. This is because a parameter matrix can continue to do product: $Para(O) = \alpha * Para'(O)$ while $mp_i = \mu_i * Para(O) = \mu_i * \alpha * Para'(O) = \mu_i * \alpha' * Para''(O)$, etc. This makes an extremely low probability of deriving the true parameters ($Para(O)$) because the decision space falls into an entire arithmetic domain.

Hence, we prove that continuously monitoring the inference process to collect more observations of morphed parameters does not increase the (low) attack probability of deriving the true parameters and scalars.

5.2 Inference Integrity

Attack assumption. Recall that we assume a third-party attacker that can hijack the client and manipulate any data outside enclave to perturb the inference process (§3.2). Specifically, we consider a *random perturbation attack* that randomly perturbs any observed data outside enclave. Next, we give analytical and experimental evaluation of the integrity guarantees of our *oblivious fingerprint* technique.

Hidden architecture for stealthiness. SOTER hides fingerprint challenges among ordinary queries to make detection stealthy. To achieve stealthiness, SOTER partitions a model by reserving *top-W* operators in enclave (Line 5 in Algorithm 2). By doing so, given a query, we hide the architecture information such that the adversary cannot determine the IRs (intermediate results transmitted from enclave to GPU) for every partitioned operator; If not doing so, assuming the first operator O_0 is partitioned to GPU, then for arbitrary query q, the adversary can get the IR of O_0 by calculating $O_0(q)$. Other IRs transmitted to O_0 are easily detected as fingerprint challenges, and the adversary would bypass our detection.



(a) Oblivious fingerprint. (b) Fixed fingerprint.

Figure 7: The l_2 distance distribution of randomly sampled IRs with oblivious or fixed fingerprints.

Fingerprint obliviousness evaluation. Even with stealthy fingerprint challenges, an adversary who monitors the entire inference process can still distinguish fixed challenges used for detection. Next, we evaluate the obliviousness of SOTER's dynamically derived fingerprints by analyzing the distribution of randomly sampled input to a given partitioned operator O.

Inspired from previous work [39], we analyzed the l_2 distance of two randomly sampled input (including both fingerprint challenges and IRs) sent from enclave to GPU, which should fits a normal (Gaussian) distribution [64, 80]. This is because, the IRs produced by a model are typically defined in a bounded space, i.e., elements of inputs are defined over a certain range of values [39]. Since we exclude black-box attacks here and assume queries are benign (§3.2), the input to O should be distributed in a natural and consistent manner.

Figure 7 depicts the l_2 distance distribution of SOTER's oblivious fingerprints and the straw-man fixed fingerprints. We observed that by using the associativity property to dynamically derive new fingerprints at runtime, the adversary can not distinguish the fingerprint challenges because the distance distribution between two random input is close to a normal distribution. Whereas, with fixed fingerprints, the adversary can observe a totally different distribution, which implies that fingerprint challenges exist and the adversary could stop attacks and evade our detection.

Proof of high detection probability. Next we prove our high probability to detect random perturbation attacks.

Denote a *perturbed* partitioned operator as O, the detection rate (probability of sending integrity challenges to O) as p, and the number of client input (queries) as N. Given that the adversary can only randomly guess whether an input to O is a challenge or not (due to the obliviousness property proved above), the probability of successfully detecting O's integrity breach after N queries is therefore $P(X = N) = 1 -$

$(1-t/2)^N$ (see Appendix A). With detection rate $p = 0.8$, SOTER can detect integrity breaches on partitioned operator with an overwhelming 99.39% probability of only $N = 10$ queries. This proof matches our evaluation in Figure 6.3.

6 Evaluation

Testbed. We conducted all evaluation in our cluster on a GPU farm of 10 DELL R430 servers with 20 Nvidia RTX2080TI graphic cards. Each graphic card is connected to the host with PCIe 3.0. Each server is equipped with an Intel 2.60GHz E5-2690 V3 CPU, 64GB memory, and 40Gbps NIC. We ran SGX with the real Intel SGX hardware in our cluster.

Model and dataset. As shown in Table 2, we evaluated nine popular DNN models include a Multi-layer Perception (MLP) model, six Convolution Neural Network (CNN) models including AlexNet [44], DenseNet-121 [36], VGG-19 [78], ResNet-101 [34], NASNet [98] and PNASNet [51], a Recurrent Neural Network (RNN) WaveRNN [41] and a transformer model [2]. Among these models, the MLP model is purely composed by the same linear operators (*torch.nn.Linear*) and ReLU activation functions (*torch.nn.ReLU*); For the CNN models, AlexNet, DenseNet-121, VGG-19, and ResNet-101 are popular models that achieve high inference accuracy on computer vision tasks. In addition to these CNN models, we evaluated two large CNN models NASNet and PNASNet which are designed by recent *neural architecture search* (NAS) techniques [58]; WaveRNN is an efficient RNN model developed by Deepmind and Google for efficient speech-to-text transformations [41]; Transformer is the most popular model for natural language processing [2].

The parameters of the models were trained by using well-known datasets on PyTorch. We used ImageNet [14] for CV models (Image Classification tasks) and WMT16 EN-DE [1] for NLP models (Translation tasks), both of which are well-known large-scale training samples.

Baselines. We compared SOTER with MLcapsule [33], a popular SGX-based secure inference system at the client-side. MLcapsule shields the entire model in SGX and executes inference tasks only with CPU, providing a fully black-box guarantee to offloaded models but coming at a substantial cost on inference latency. Since MLcapsule is not open-sourced, we built a proof-of-concept implementation based on the descriptions in MLcapsule's paper [33]. We also compared SOTER with the state-of-the-art model partition approach eNNclave [70], a notable open-sourced inference system at the client-side that uses handcrafted model partition. Although DeepEnclave [32] uses SGX for model partition, we did not evaluate it because DeepEnclave has an orthogonal focus on protecting client's input data.

We divide our evaluation into three sections that answer the following questions:

§6.1 How efficient is SOTER compared with existing secure inference systems at the client-side?

§6.2 How accurate is SOTER compared with existing secure inference systems at the client-side?

§6.3 How robust is SOTER under integrity attacks?

6.1 End-to-end Performance

End-to-end inference latency. We first investigated the end-to-end inference latency of SOTER and two baseline systems with nine models. All reported measurements are the average inference latency of 20 independent inferences on the same input. As shown in Figure 8, the partition-based secure inference systems including SOTER (execute inference tasks with both CPU and GPU) achieved better performance than MLcapsule (executes inference tasks only with CPU). This is because the CPU is at least one order of magnitude less efficient than GPU [83] in executing the inference tasks. Among all baseline systems, SOTER achieved the lowest inference latency for all models. Moreover, the latency gaps between SOTER and baseline systems grew larger as model / input size increases (shown in Figure 9). For instance, with a small DNN model (AlexNet), SOTER achieved $\sim 3.0\%$ lower inference latency on average than existing secure inference systems. With a larger DNN model (Transformer), SOTER achieved up to 22.6% lower inference latency on average than existing secure inference systems.

Latency breakdown. To understand SOTER's low inference latency, we broke down the time taken in different phases of an inference task in SOTER, and the results are reported in Table 3. It can be clearly observed that, for all models, with a 80% probability ($\theta = 0.8$) to partition an associative operator, SOTER securely offloaded most computation-expensive model layers to GPU and successfully downgraded inference latency compared to pure CPU inference, resulting in low end-to-end inference latency. Doing so requires SOTER to copy the model layer parameters from SGX to the GPU memory only in the preprocessing phase. During the inference phase, SOTER only copies intermediate results (much smaller than the model parameters) into the GPU memory, and copies the outputs back to the SGX enclave memory for each layer. Frequent memory copies between GPU and SGX may cause a substantial latency penalty. To amortize these memory copy overhead, SOTER batches several continuous operators together, and performs input/output memory copies only before and after computing the batch of operators.

Overall, SOTER can achieve a low inference latency, and the computation overhead (CPU computation in SGX and kernel switch) caused by enclaving the plaintext operators only increases sub-linearly with the model size.

6.2 Inference Accuracy

In this section, we evaluate the resulting inference accuracy of SOTER and existing secure inference systems. For each system, we compared the inference accuracy of the protected

Table 2: Overview of evaluated models and datasets.

Task	Pattern	Model	Param Size (MB)	Defaulte Input Tensor Shape	Dataset
Image Classification	MLP	MNIST-MLP (MLP) [46]	4.58	(1, 1, 28, 28)	MNIST [45]
	CNN	AlexNet (AN) [44]	233.08	(1, 3, 224, 224)	ImageNet [14]
		DenseNet-121 (DN) [36]	30.44		
		VGG-19 (VGG) [78]	548.05		
		ResNet-101 (RN) [34]	169.94		
		NASNet (NN) [98]	338.57	(1, 3, 331, 331)	
		PNASNet (PNN) [51]	328.68		
Translation	RNN	WaveRNN (WR) [41]	1254.64	512 tokens	WMT16 (En-De) [1]
	Transformer	Nvidia-Transformer (TF) [2]	1335.09	512 tokens	WMT16 (En-De) [1]

Table 3: Latency breakdown of SOTER ($\theta = 0.8$).

Models	Preprocessing (s)	Inference (ms)		
		SGX (CPU)	Kernel Switch	GPU
MLP	0.32	0.08	0.005	0.001
AlexNet	1.56	3.475	0.02	0.76
VGG-19	3.52	4.82	0.05	3.64
ResNet-101	2.29	2.30	0.06	7.10
DenseNet121	2.23	1.03	0.11	4.77
NASNet	8.23	274.43	11.21	27.71
PNASNet	9.47	293.61	12.38	29.41
WaveRNN	11.51	312.25	15.13	26.35
Transformer	12.41	334.97	15.60	34.35

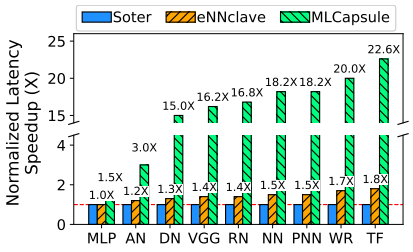


Figure 8: Normalized end-to-end inference latency comparison.

models (i.e., running some or all layers of the ML model in SGX enclaves to protect the model) and the corresponding white-box unprotected model (i.e., running the entire model on GPU without the protection of SGX).

As shown in Figure **??**, SOTER and MLcapsule achieved zero accuracy drop, while other partition-based systems caused inference accuracy drops from 1% to 5.5% for all evaluated CNN models. MLcapsule was able to achieve zero accuracy loss because it shields the entire model in the SGX enclave to protect the highest security guarantee. However, at the same time, the security guarantee of MLcapsule come at a steep price in performance, compared to untrusted alternatives (i.e., perform inference tasks on the GPU).

To achieve better performance, the partition-based secure inference systems outsource some inference computations to the untrusted GPU. However, existing partition-based inference system (i.e., eNNclave) takes a handcrafted approach to partition an offloaded model and tunes the parameters of the outsourced layers, leading to a drop in the inference accuracy. By safely morphing and partitioning computation-expensive operators to GPU for faster inference computing, and then

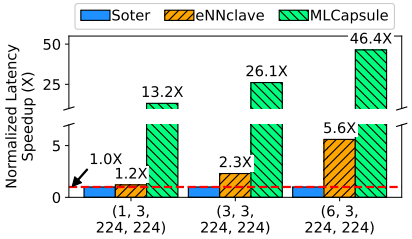


Figure 9: Inference latency with different input shape (batch_sise, channels, height, weight).

recovers the real inference result in SGX, SOTER achieves stronger model confidentiality (§5.1) compared to eNNclave without sacrificing the inference accuracy.

6.3 Robustness to Integrity Breach Attacks

We evaluated the robustness of SOTER under the integrity breach attacks. We conducted an integrity attack by continuously and randomly tempering with the outputs of layers outside the enclave, and monitoring the number of fingerprints for SOTER to detect this attack. The evaluation results of 10K attacks are shown in Figure 10. All attacks were successfully detected, and 99.1% attacks were detected with less than 10 fingerprints while we incurred zero false positives. This result matches our theoretical analysis in §5.2.

7 Discussion

7.1 TEE-based handcrafted approaches

AI applications (e.g., face recognition [48], autopilot [8]) require both high performance (i.e., low inference latency and high accuracy) and strong privacy. Due to the lack of commonly-available trusted GPUs [5, 87], many existing systems propose protocols to empower secure model inference by co-designing an accelerator (e.g., GPU) with Trusted Execution Environments (e.g., Intel SGX [12]).

eNNclave [70] is a state-of-the-art secure inference system that engages the power of both accelerator and TEE. It has the advantage of low inference latency and even achieves a stronger privacy guarantee for specific models compared to SOTER (as shown in Figure 6). However, unlike SOTER, eNNclave handcrafts a partition plan by manually identify-

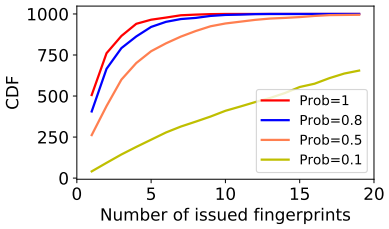


Figure 10: The CDF of the number of issued fingerprint challenges for catching 1000 random perturbation attacks with different issuing probability P.

ing less sensitive model layers (operators), and then uses transfer learning techniques [72] to retrain these layers' parameters into function-similar, yet insensitive new parameters. Unfortunately, such handcrafted approach inevitably meets a generality bottleneck. Specifically, unlike traditional layered DNNs (e.g., VGG series models) that have clear sensitivity boundaries between layers [37, 81, 92], modern DNNs are increasingly designed with complex architectures (composing complicated linkages among numerous operators [47, 62, 97]) for higher accuracy. Due to the heuristic nature (weak explainability) of complex models [21, 66, 93], manually determining the sensitivity for every operator (layer) could be extremely hard. If falsely selects a sensitive layer and retrains it with transfer learning, the inference accuracy would severely downgrades, leading to unexpected failures of applications.

DeepEnclave [32], like eNNclave, is another handcrafted model partition approach for secure inference. It uses knowledge distillation [11, 94] to hide manually identified sensitive layers. However, DeepEnclave is a cloud-side inference system with a totally different focus on protecting users' input queries, hence SOTER does not compare with DeepEnclave.

7.2 Limitations and Future Work

Model leakage from black-box attacks. Modern black-box attacks [6, 39, 59, 61] use inference APIs of an inferene service to learn private information of an offloaded model or even the training dataset, by means of the inference results of perturbed input queries. These attacks widely exist in either cloud-side or client-side model inference systems [33, 40, 55, 70], as inference systems inevitably open access to arbitrary queries.

Currently, there is no general defenses against black-box attacks other than query authentication and rate limiting [39, 59]. But fortunately, defenses against specifc attacks are emerging. For instance, Prada [39] uses query distance auditing to mitigate model extraction attacks with adversarial examples; Shokroi et al. [77] uses differential privacy to train DNNs that do not leak model owner's sensitive training dataset.

The guarantees of SOTER, like all prior secure inference systems targeting at protecting the model at the edge or the cloud, are complementary to those provided by any defenses. With sufficient engineering effort, these mitigations can be integrated into SOTER to provide even stronger privacy guarantees, and we leave it to future work.

Usage of Trusted Execution Environment. SOTER requires clients at the edge to be equipped with TEE (e.g., Intel SGX) due to the lack of commonly available trusted GPUs [5, 87]. SGX has been pervasively used in existing secure inference systems [33, 70] and is commonly available in modern Intel CPUs. While the inherited SGX vulnerabilities of sophisticated side-channel attacks based on timing or cache access patterns still exist [15, 88], we do not consider these attacks currently, and we believe these attacks can be mitigated by integrating with state-of-the-art defenses [15, 16, 68].

8 Conclusion

We present SOTER, the first secure inference protocol that ensures both strong confidentiality and integrity for general DNN models. SOTER partitions computation-expensive operators from TEE to the co-located untrusted GPU by morphing a fraction of associative operators' parameters and recovers the real execution results in TEE. Evaluation on notable client-side secure inference systems and all prevalent types of DNN models shows that SOTER is the first protocol that can achieve low inference latency while maintaining high accuracy at the same time. SOTER is general for placing proprietary DNN models on untrustworthy clients at the edge, and is especially suitable for security-critical and performance-sensitive AI applications such as autopilot and stock prediction. SOTER's source code is released on `github.com/ndss22p16/soter`.

Appendices

A SOTER's Detection probability

The contents...

References

[1] Wmt16. http://www.statmt.org/wmt16/translation-task.html, 2016.

[2] Transformer. https://ngc.nvidia.com/catalog/resources, 2020.

[3] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.

[4] Anshul Aggarwal, Trevor E Carlson, Reza Shokri, and Shruti Tople. Soteria: In search of efficient neural networks for private inference. *arXiv preprint arXiv:2007.12934*, 2020.

[5] Aref Asvadishirehjini, Murat Kantarcioglu, and Bradley Malin. Goat: Gpu outsourcing of deep learning training

with asynchronous probabilistic integrity verification inside trusted execution environment. *arXiv preprint arXiv:2010.08855*, 2020.

[6] Buse Gul Atli, Sebastian Szyller, Mika Juuti, Samuel Marchal, and N Asokan. Extraction of complex dnn models: Real threat or boogeyman? In *International Workshop on Engineering Dependable and Secure Machine Learning Systems*, pages 42–57. Springer, 2020.

[7] J Aumasson and L Merino. Sgx secure enclaves in practice–security and crypto review. *Black Hat*, 2016.

[8] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[9] Han Cai, Ligeng Zhu, and Song Han. Proxylessnas: Direct neural architecture search on target task and hardware. *arXiv preprint arXiv:1812.00332*, 2018.

[10] Tiago Carvalho, Edmar RS De Rezende, Matheus TP Alves, Fernanda KC Balieiro, and Ricardo B Sovat. Exposing computer generated images by eye's region classification via transfer learning of vgg19 cnn. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 866–870. IEEE, 2017.

[11] Xu Cheng, Zhefan Rao, Yilan Chen, and Quanshi Zhang. Explaining knowledge distillation by quantifying the knowledge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12925–12935, 2020.

[12] Victor Costan and Srinivas Devadas. Intel sgx explained. *IACR Cryptol. ePrint Arch.*, 2016(86):1–118, 2016.

[13] Piali Das, Nikita Ivkin, Tanya Bansal, Laurence Rouesnel, Philip Gautier, Zohar Karnin, Leo Dirac, Lakshmi Ramakrishnan, Andre Perunicic, Iaroslav Shcherbatyi, et al. Amazon sagemaker autopilot: a white box automl solution at scale. In *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, pages 1–7, 2020.

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[15] Ghada Dessouky, Tommaso Frassetto, and Ahmad-Reza Sadeghi. Hybcache: Hybrid side-channel-resilient caches for trusted execution environments. In *29th {USENIX} Security Symposium ({USENIX} Security 20)*, pages 451–468, 2020.

[16] Anuj Dubey, Rosario Cammarota, Vikram Suresh, and Aydin Aysu. Guarding machine learning hardware against physical side-channel attacks. *arXiv preprint arXiv:2109.00187*, 2021.

[17] S Durga, Esther Daniel, and S Deepakanmani. Dnn-based decision support system for ecg abnormalities. In *2nd EAI International Conference on Big Data Innovation for Sustainable Cognitive Computing*, pages 331–338. Springer, 2021.

[18] Cynthia Dwork. Differential privacy: A survey of results. In *International conference on theory and applications of models of computation*, pages 1–19. Springer, 2008.

[19] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *The Journal of Machine Learning Research*, 20(1):1997–2017, 2019.

[20] Simon Fong, Suash Deb, and Xin-she Yang. How meta-heuristic algorithms contribute to deep learning in the hype of big data analytics. In *Progress in intelligent computing techniques: theory, practice, and applications*, pages 3–25. Springer, 2018.

[21] Krishna Gade, Sahin Cem Geyik, Krishnaram Kenthapadi, Varun Mithal, and Ankur Taly. Explainable ai in industry. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3203–3204, 2019.

[22] Yongsheng Gao and Maylor KH Leung. Face recognition using line edge map. *IEEE transactions on pattern analysis and machine intelligence*, 24(6):764–779, 2002.

[23] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International Conference on Machine Learning*, pages 201–210. PMLR, 2016.

[24] Alessandro Giusti, Dan C Cireşan, Jonathan Masci, Luca M Gambardella, and Jürgen Schmidhuber. Fast image scanning with deep max-pooling convolutional neural networks. In *2013 IEEE International Conference on Image Processing*, pages 4034–4038. IEEE, 2013.

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[26] Ian Goodfellow, Patrick McDaniel, and Nicolas Papernot. Making machine learning robust against adversarial inputs. *Communications of the ACM*, 61(7):56–66, 2018.

[27] Google. Cloud speech-to-text api -language support. https://cloud.google.com/speech-to-text/docs/languages.

[28] Google. Google lens. https://lens.google/.

[29] T Gopalakrishnan, D Ruby, Fadi Al-Turjman, Deepak Gupta, Irina V Pustokhina, Denis A Pustokhin, and K Shankar. Deep learning enabled data offloading with cyber attack detection model in mobile edge computing systems. *IEEE Access*, 8:185938–185949, 2020.

[30] Thore Graepel, Kristin Lauter, and Michael Naehrig. Ml confidential: Machine learning on encrypted data. In *International Conference on Information Security and Cryptology*, pages 1–21. Springer, 2012.

[31] Luca Greco, Pierluigi Ritrovato, and Fatos Xhafa. An edge-stream computing infrastructure for real-time analysis of wearable sensors data. *Future Generation Computer Systems*, 93:515–528, 2019.

[32] Zhongshu Gu, Heqing Huang, Jialong Zhang, Dong Su, Ankita Lamba, Dimitrios Pendarakis, and Ian Molloy. Securing input data of deep learning inference systems via partitioned enclave execution. *arXiv preprint arXiv:1807.00969*, 2018.

[33] Lucjan Hanzlik, Yang Zhang, Kathrin Grosse, Ahmed Salem, Maximilian Augustin, Michael Backes, and Mario Fritz. Mlcapsule: Guarded offline deployment of machine learning as a service. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3300–3309, 2021.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[35] David A Hensher and William H Greene. The mixed logit model: the state of practice. *Transportation*, 30(2):133–176, 2003.

[36] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

[37] Joanna Jaworek-Korjakowska, Pawel Kleczek, and Marek Gorgon. Melanoma thickness prediction based on convolutional neural network with vgg-19 model transfer learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 0–0, 2019.

[38] Jianyu Jiang, Xusheng Chen, TszOn Li, Cheng Wang, Tianxiang Shen, Shixiong Zhao, Heming Cui, Cho-Li Wang, and Fengwei Zhang. Uranus: Simple, efficient sgx programming and its applications. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, pages 826–840, 2020.

[39] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N Asokan. Prada: protecting against dnn model stealing attacks. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 512–527. IEEE, 2019.

[40] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. {GAZELLE}: A low latency framework for secure neural network inference. In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, pages 1651–1669, 2018.

[41] Nal Kalchbrenner, Erich Elsen, Karen Simonyan, Seb Noury, Norman Casagrande, Edward Lockhart, Florian Stimberg, Aaron Oord, Sander Dieleman, and Koray Kavukcuoglu. Efficient neural audio synthesis. In *International Conference on Machine Learning*, pages 2410–2419. PMLR, 2018.

[42] Rafał Kozik, Michał Choraś, Massimo Ficco, and Francesco Palmieri. A scalable distributed machine learning approach for attack detection in edge computing environments. *Journal of Parallel and Distributed Computing*, 119:18–26, 2018.

[43] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[44] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012.

[45] Yann LeCun. The mnist database of handwritten digits. *http://yann. lecun. com/exdb/mnist/*, 1998.

[46] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[47] Yuan-Shan Lee, Chien-Yao Wang, Shu-Fan Wang, Jia-Ching Wang, and Chung-Hsien Wu. Fully complex deep neural network for phase-incorporating monaural source separation. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 281–285. IEEE, 2017.

[48] Jinyu Li, Dong Yu, Jui-Ting Huang, and Yifan Gong. Improving wideband speech recognition using mixed-bandwidth training data in cd-dnn-hmm. In *2012 IEEE Spoken Language Technology Workshop (SLT)*, pages 131–136. IEEE, 2012.

[49] Qianmu Li, Shunmei Meng, Sainan Zhang, Jun Hou, and Lianyong Qi. Complex attack linkage decision-making in edge computing networks. *IEEE access*, 7:12058–12072, 2019.

[50] Yandong Li, Lijun Li, Liqiang Wang, Tong Zhang, and Boqing Gong. Nattack: Learning the distributions of adversarial examples for an improved black-box attack on deep neural networks. In *International Conference on Machine Learning*, pages 3866–3876. PMLR, 2019.

[51] Chenxi Liu, Barret Zoph, Maxim Neumann, Jonathon Shlens, Wei Hua, Li-Jia Li, Li Fei-Fei, Alan Yuille, Jonathan Huang, and Kevin Murphy. Progressive neural architecture search. In *Proceedings of the European conference on computer vision (ECCV)*, pages 19–34, 2018.

[52] Yanpei Liu, Xinyun Chen, Chang Liu, and Dawn Song. Delving into transferable adversarial examples and black-box attacks. *arXiv preprint arXiv:1611.02770*, 2016.

[53] Xiaolong Ma, Fu-Ming Guo, Wei Niu, Xue Lin, Jian Tang, Kaisheng Ma, Bin Ren, and Yanzhi Wang. Pconv: The missing but desirable sparsity in dnn weight pruning for real-time execution on mobile devices. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5117–5124, 2020.

[54] Lubin Meng, Chin-Teng Lin, Tzyy-Ping Jung, and Dongrui Wu. White-box target attack for eeg-based bci regression problems. In *International conference on neural information processing*, pages 476–488. Springer, 2019.

[55] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference service for neural networks. In *29th {USENIX} Security Symposium ({USENIX} Security 20)*, pages 2505–2522, 2020.

[56] Mohammad H Mofrad, Adam Lee, and Spencer L Gray. Leveraging intel sgx to create a nondisclosure cryptographic library. *arXiv preprint arXiv:1705.04706*, 2017.

[57] Robert C Moore and Will Lewis. Intelligent selection of language model training data. 2010.

[58] Stefan Nastic, Thomas Rausch, Ognjen Scekic, Schahram Dustdar, Marjan Gusev, Bojana Koteska, Magdalena Kostoska, Boro Jakimovski, Sasko Ristov, and Radu Prodan. A serverless real-time data analytics platform for edge computing. *IEEE Internet Computing*, 21(4):64–71, 2017.

[59] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Prediction poisoning: Towards defenses against dnn model stealing attacks. *arXiv preprint arXiv:1906.10908*, 2019.

[60] Daniele Palossi, Antonio Loquercio, Francesco Conti, Eric Flamand, Davide Scaramuzza, and Luca Benini. A 64-mw dnn-based visual navigation engine for autonomous nano-drones. *IEEE Internet of Things Journal*, 6(5):8357–8371, 2019.

[61] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pages 506–519, 2017.

[62] Adam Paszke, Abhishek Chaurasia, Sangpil Kim, and Eugenio Culurciello. Enet: A deep neural network architecture for real-time semantic segmentation. *arXiv preprint arXiv:1606.02147*, 2016.

[63] Li Pengcheng, Jinfeng Yi, and Lijun Zhang. Query-efficient black-box attack by active learning. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 1200–1205. IEEE, 2018.

[64] Johan Philip. *The probability distribution of the distance between two random points in a box*. KTH mathematics, Royal Institute of Technology, 2007.

[65] Matt Post. A call for clarity in reporting bleu scores. *arXiv preprint arXiv:1804.08771*, 2018.

[66] Alun Preece. Asking 'why'in ai: Explainability of intelligent systems–perspectives and challenges. *Intelligent Systems in Accounting, Finance and Management*, 25(2):63–72, 2018.

[67] Hussam Qassim, Abhishek Verma, and David Feinzimer. Compressed residual-vgg16 cnn model for big data places image recognition. In *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 169–175. IEEE, 2018.

[68] Sajin Sasy, Sergey Gorbunov, and Christopher W Fletcher. Zerotrace: Oblivious memory primitives from intel sgx. In *NDSS*, 2018.

[69] Takami Sato, Junjie Shen, Ningfei Wang, Yunhan Jack Jia, Xue Lin, and Qi Alfred Chen. Security of deep learning based lane keeping system under physical-world adversarial attack. *arXiv preprint arXiv:2003.01782*, 2020.

[70] Alexander Schlögl and Rainer Böhme. ennclave: Offline inference with model confidentiality. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, pages 93–104, 2020.

[71] Rico Sennrich, Barry Haddow, and Alexandra Birch. Edinburgh neural machine translation systems for wmt 16. *arXiv preprint arXiv:1606.02891*, 2016.

[72] Mehmet Saygin Seyfioglu, Baris Erol, Sevgi Zubeyde Gurbuz, and Moeness G Amin. Dnn transfer learning from diversified micro-doppler for motion classification. *IEEE Transactions on Aerospace and Electronic Systems*, 55(5):2164–2180, 2018.

[73] Jiawei Shao and Jun Zhang. Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems. In *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*, pages 1–6. IEEE, 2020.

[74] Tianxiang Shen, Jianyu Jiang, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, and Heming Cui. Daenet: Making strong anonymity scale in a fully decentralized network. *IEEE Transactions on Dependable and Secure Computing*, 2021.

[75] Youren Shen, Hongliang Tian, Yu Chen, Kang Chen, Runji Wang, Yi Xu, Yubin Xia, and Shoumeng Yan. Occlum: Secure and efficient multitasking inside a single enclave of intel sgx. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 955–970, 2020.

[76] Kilho Shin and Tetsuji Kuboyama. A generalization of haussler's convolution kernel: mapping kernel. In *Proceedings of the 25th international conference on Machine learning*, pages 944–951, 2008.

[77] Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1310–1321, 2015.

[78] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[79] Erik LL Sonnhammer, Gunnar Von Heijne, Anders Krogh, et al. A hidden markov model for predicting transmembrane helices in protein sequences. In *Ismb*, volume 6, pages 175–182, 1998.

[80] Luis Antonio Santaló Sors and Luis A Santaló. *Integral geometry and geometric probability*. Cambridge university press, 2004.

[81] Srikanth Tammina. Transfer learning using vgg-16 with deep convolutional neural network for classifying images. *International Journal of Scientific and Research Publications (IJSRP)*, 9(10):143–150, 2019.

[82] Sasha Targ, Diogo Almeida, and Kevin Lyman. Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*, 2016.

[83] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287*, 2018.

[84] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204*, 2017.

[85] Tuyen X Tran, Mohammad-Parsa Hosseini, and Dario Pompili. Mobile edge computing: Recent efforts and five key research directions. *IEEE COMSOC MMTC Commun.-Frontiers*, 2017.

[86] Chia-Che Tsai, Donald E Porter, and Mona Vij. Graphene-sgx: A practical library {OS} for unmodified applications on {SGX}. In *2017 {USENIX} Annual Technical Conference ({USENIX}{ATC} 17)*, pages 645–658, 2017.

[87] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. Graviton: Trusted execution environments on gpus. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pages 681–696, 2018.

[88] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2421–2434, 2017.

[89] Wenxuan Wang, Bangjie Yin, Taiping Yao, Li Zhang, Yanwei Fu, Shouhong Ding, Jilin Li, Feiyue Huang, and Xiangyang Xue. Delving into data: Effectively substitute training for black-box attack. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4761–4770, 2021.

[90] Yajie Wang, Yu-an Tan, Wenjiao Zhang, Yuhang Zhao, and Xiaohui Kuang. An adversarial attack on dnn-based black-box object detectors. *Journal of Network and Computer Applications*, 161:102634, 2020.

[91] Jongho Won, Seung-Hyun Seo, and Elisa Bertino. A secure shuffling mechanism for white-box attack-resistant unmanned vehicles. *IEEE Transactions on Mobile Computing*, 19(5):1023–1039, 2019.

[92] Yong Wu, Xiao Qin, Yonghua Pan, and Changan Yuan. Convolution neural network based transfer learning for classification of flowers. In *2018 IEEE 3rd international conference on signal and image processing (ICSIP)*, pages 562–566. IEEE, 2018.

[93] Roman V Yampolskiy. Unexplainability and incomprehensibility of ai. *Journal of Artificial Intelligence and Consciousness*, 7(02):277–291, 2020.

[94] Junho Yim, Donggyu Joo, Jihoon Bae, and Junmo Kim. A gift from knowledge distillation: Fast optimization, network minimization and transfer learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4133–4141, 2017.

[95] Chao Zhang and Philip C Woodland. Parameterised sigmoid and relu hidden activation functions for dnn acoustic modelling. In *Sixteenth Annual Conference of the International Speech Communication Association*, 2015.

[96] Qiao Zhang, Chunsheng Xin, and Hongyi Wu. Gala: Greedy computation for linear algebra in privacy-preserved neural networks. *arXiv preprint arXiv:2105.01827*, 2021.

[97] Zhimian Zhang, Haipeng Wang, Feng Xu, and Ya-Qiu Jin. Complex-valued convolutional neural network and its application in polarimetric sar image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 55(12):7177–7188, 2017.

[98] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8697–8710, 2018.

[99] Pengfei Zuo, Yu Hua, Ling Liang, Xinfeng Xie, Xing Hu, and Yuan Xie. Sealing neural network models in secure deep learning accelerators. *arXiv preprint arXiv:2008.03752*, 2020.